A level and a session

A level and a session

Graal Analysis, revision 0.0.2.

A level is a square board of tiles, plus the lists that sit on that board. A session is what a client sends once it has somewhere to stand. This chapter is those two things, in that order. The working notes stay under `formats/`, `protocol/`, and `tools/level-convert/`.

Where two notes give two figures, this chapter says that they do and leaves the pair in chapter 4.

A square of tiles

The board is 64 cells on a side. Cell `(x, y)` is stored at index `x + 64*y`, row by row, and a run has to stay inside x from 0 to 63 (`formats/glevnw01.txt`). Each cell is 16 pixels on the standard sheet (`formats/tiletypes.txt`). The salvage note and the later Seihai board note both call the resulting world 1024 pixels on a side (`protocol/format-notes-from-salvage.md`, `formats/seihai-boards.md`). The Seihai note is a different game that reused the size. The 64 by 64 grid is the Graal one.

The number in a cell is a tile id, from 0 through 4095. It is an index into the tile sheet. On `pics1.png` the sheet is 2048 by 512 pixels, a grid of 128 by 32 tiles. Column is the id modulo 128, row is the id divided by 128, and the picture starts at 16 times that column and row (`formats/tiletypes.txt`).

The client decides a step from tables loaded beside the picture. The notes describe more than one such table, and they do not give the same counts. Chapter 4 quotes them. The file itself stores the id.

The eight bytes at the front of a file say which kind of file it is. A name ending in `.graal` can hold a Z3-V1 file, and a name ending in `.zelda` can hold a GR-V1 file (`formats/level-binary.md`).

The magics the 2.05 string table holds are `Z3-V1.03`, `Z3-V1.04`, `GR-V1.00` through `GR-V1.03`, and `GLEVNW01`. Each of those magics appears three times in the string table, and one copy sits next to the web-level cache code. The 2.05 loader itself had not been traced when the server-source note was written, so the bit rules below were taken from the server (`formats/level-binary.md`). A later read of the 2.05 and 2.31 clients agrees with those rules on the points named in `formats/level-binary-client.md`.

How the board is packed

The same 4096 tile ids have been written three different ways.

The binary board

`GR-V1.00` through `GR-V1.03`, and the Z3-V1 files, store the board as a packed bit stream with run-length codes. An older summary called GR-V1 a 64 by 64 board of 2-byte tiles. `formats/level-binary.md` withdraws that for the file. The text level and the full-board packet are the forms that store one tile id per cell. Those two forms are described later. Chapter 4 keeps the old sentence beside the withdrawal.

The code width depends on the magic.

Magic	Bits in each code	Where that is stated
`Z3-V1.02`	11	`formats/level-binary.md`. Only the 1998 Java Zelda Online games read this file.
`GR-V1.00`, `GR-V1.01`, `Z3-V1.03`, `Z3-V1.04`	12	`formats/level-binary.md`
`GR-V1.02`, `GR-V1.03`, and `GR-V1.04` in 2.31	13	`formats/level-binary.md`, `formats/level-binary-client.md`

`formats/level-binary-client.md` says the 2.05 and 2.31 readers use 12 bits for both `Z3-V1.*` versions they handle. It does not mention `Z3-V1.02`. Read as a claim about `.03` and `.04`, it matches the server note. Read as a claim about every Z3 file, it does not. The two sentences are quoted in chapter 4. The converter follows the server note: it reads `Z3-V1.02` as 11-bit codes, and when it writes a `.zelda` file it writes `Z3-V1.04` (`tools/level-convert/README.md`).

Codes are taken least-significant bit first. Bytes are appended to a little-endian bit buffer, and `width` bits are taken from the bottom. The top bit of the code is a flag. The low eight bits are a count. The bit just above those eight bits, when the flag is set, means the next run is a double run (`formats/level-binary.md`).

When the flag is clear, the code is a tile id. The count starts at 1, and a count of 1 writes that tile once. A single run writes the next tile `count` times, then sets the count back to 1. A double run reads two tiles, A and B, and writes the pair A B, `count` times, then sets the count back to 1. Tiles fill layer 0 only, in the row-major order above.

A 12-bit code cannot name a tile of 2048 or above, because the flag uses the top bit. The widening to 13 bits in `GR-V1.02` is what makes room for a 4096-tile `pics1`. That reason is marked inferred (`formats/level-binary.md`).

The 2.05 writer walks the 4096 tiles and emits a single run for 2 to 64 equal tiles, or a double run of up to 64 pairs where an A B A B pattern starts, and otherwise the tile itself. For a file that is not `.nw`, that writer always emits `GR-V1.03`. Single runs of exactly 64 tiles occur in the embedded tutorial levels (`formats/level-binary.md`, `formats/level-binary-client.md`). Re-encoding the boards of 144 cached levels and 13 embedded tutorial levels reproduced every board byte. `[2.05]` `[2.31]` `[test]`

After the board, a binary file continues in a fixed order (`formats/level-binary.md`).

1. Link lines, until a `#` or an empty line.
2. Baddy records, until a terminator, each followed by a verse line. `Z3-V1.03` has no verse line.
3. NPC lines, until a `#` or an empty line. This section is absent from `.zelda`.
4. Chest lines, until a `#`. Present from `GR-V1.01` on, and absent from `.zelda`.
5. Sign lines, until an empty line or the end of the file.

The 2.05 client reads those sections with the same version gates: links for every version, baddies for every version, NPCs from `GR-V1.00`, chests from `GR-V1.01`, and signs as whatever is left. A baddy record is read up to a newline that comes after at least three bytes, so one of those three bytes may itself be a newline. The client stops the baddy list at a record whose first byte is 128 or higher, which is how it sees the `FF FF FF` terminator (`formats/level-binary-client.md`).

Signs are the point where 2.05 and 2.31 part. 2.05 reads signs through to the end of the file. 2.31 stops at the first empty line, because a height block can follow (`formats/level-binary-client.md`). The server-source layout already said "empty line or end of file". Both client behaviours fit that sentence. They do not fit each other if a height block is present. Chapter 4 leaves that case open.

`GR-V1.04` is `GR-V1.03` plus an optional height map. The 2.31 writer emits `GR-V1.04` only when the level has one, and `GR-V1.03` otherwise. After the signs it writes a blank line, the word `HEIGHTS`, nine lines of nine comma-

separated numbers, and HEIGHTSEND. The reader accepts that block after the signs whatever the magic says, matches HEIGHTSEND without caring about case, and keeps at most nine rows of nine values. The board coding stays 13-bit. In 2.31 this parse shows up only in the web-level cache, which is the one place that client both writes and reads GR-V1.04. [2.31] (formats/level-binary-client.md). A 9 by 9 grid lines up with the corners of an 8 by 8 grid of 8 by 8 cells on a 64 by 64 level. That fit is inferred.

The same HEIGHTS block is what the terrain generator writes into a GLEVNW01 file, after a board of 64 rows of tile 0 (formats/glevnw01.txt, formats/level-binary-client.md).

The text board

A .nw file is line-oriented text. The first line is GLEVNW01. Servers send a payload the same size as a binary .graal file. The copy on disk is the larger one. Official notes say to edit it in an editor that does not wrap lines (formats/glevnw01.txt).

A board line looks like BOARD, then x, y, width, a layer index, then the tiles. It is one horizontal run. The tile text is width * 2 characters. Each tile is a 12-bit index, packed as two characters from A–Z, a–z, 0–9, +, /, high six bits first. AA is tile 0. // is tile 4095. The loader ignores a BOARD line unless it has six tokens, the run stays inside x from 0 to 63, and the tile text is at least width * 2 characters. The layer index is parsed and then left unused (formats/glevnw01.txt).

The client alphabet in the loader appends a trailing =. That character is outside the 64-character alphabet above (formats/glevnw01.txt).

The other text records use the same words a binary file uses, with different spelling. A LINK line is the destination, the box, and the arrival, and the destination and the two arrival fields must not contain spaces. A SIGN is SIGN, x, y, a slash, the text, a slash, and SIGNEND. The body may be empty. An NPC is an image name (a hyphen if there is no image), x, y, a slash, the script, a slash, and NPCEND. Those x and y may be floating point. A BADDY is x, y, a type name or a number from 0 to 9, verses, and BADDYEND. The verses may be shortened. The end marker is required. A CHEST is x, y, an item name or a number from 0 to 24, and a sign index. An unknown item name drops the chest (formats/glevnw01.txt).

formats/glevnw01.txt describes GraalEditor as opening *.graal and *.nw through one filter and treating both as text, without requiring the GLEVNW01 signature. In that description, BOARD 0 is the tile layer as 64 lines of 64 decimal indices, heights are an overlay, and the section words match while the field order is the order the dialogs collect. tools/graaleditor.md says that decimal-board claim has not been re-checked against a file GraalEditor itself saved, and that it conflicts with the base64 rows the built-in 2.05 and 2.22 editors write. Chapter 4 quotes both. A file of raw indices has to be rewritten as GLEVNW01 base64 runs before the level loader described in formats/glevnw01.txt will take it.

Three writers share this neighbourhood, and two of them share the signature. The terrain generator writes GLEVNW01 with 64 rows of tile 0, a 9 by 9 HEIGHTS block, and a LINK on each edge that has a neighbour. That image has no NPC section. The dungeon generator writes GLEVNW01 with one base64 board of 64 by 64, edge links, and NPC blocks. Stamp id 0 is replaced by a 4 by 4 ground fill before the board is stored. Stamp id 1 is a hole and is not written. The decimal BOARD 0 described above is the third shape (formats/glevnw01.txt, formats/gobjset01.md).

What a conversion keeps

The converter in tools/level-convert/ walks the chain in both directions, as far as each format can carry the data: .zelda, then .graal, then .nw, then a .gmap, then a package. TextConvert, the older folder walker, writes

GLEVNW01 from GR-V1 only. It does not read Z3-V1, it does not write a gmap, and it has no package step (tools/level-convert/README.md).

Between .graal and .nw, the 64 by 64 board survives for tile ids 0 through 4095, and so do links, signs, baddies with a numeric type, NPCs, and chests whose item is one of the 25 names listed in client-internals/level-editor-205.md. Between .zelda and the later files, the board and the links survive when every tile is in 0 through 2047, plus signs and numeric baddies. A real Z3-V1 file has no NPC section and no chest section, so those lists stay empty. Tile ids above 2047, written into a .zelda file, are stored as the low 11 bits, and the tool warns. A HEIGHTS overlay is kept on .nw, and GR-V1.04 is read as GR-V1.03 plus that block, but saving a .graal file still writes GR-V1.03 and says the overlay was not written (tools/level-convert/README.md).

An NPC coordinate that is not a whole tile is kept by .nw (10.5 is the example in that note). GR-V1 stores a byte, the coordinate plus 32, so the fraction is cut off. A link destination that contains spaces is kept by GR-V1, which rejoins every token before the last six. GLEVNW01 keeps one token, and the converter does not write that link. A baddy type written as a name is kept on .nw and is not given a byte on .graal or .zelda. A numeric type from 0 to 9 round-trips (tools/level-convert/README.md, formats/level-binary.md).

formats/level-binary.md says the coordinates in these binary records are whole tiles. [srv] The same note later says Z3-V1.03 link numbers are floating point, including halves. formats/glevnw01.txt allows fractional NPC positions. The arrival count below includes halves such as 24.5. Chapter 4 sets those sentences side by side.

Binary levels are read with formats/grv1-decoder. A .graal board is written with the 13-bit GR-V1.03 writer in tools/file-formats/gwebl.py. The converter's tests are synthetic. No client program is run (tools/level-convert/README.md).

What a link is

A link is a doorway written as text.

In a binary file the line is a destination, then x y w h, then newx newy. The six numbers are the box on this level and the place the player arrives on the destination. Spaces in the destination are allowed, because every token before the last six is joined back together. A link whose destination level is missing is dropped (formats/level-binary.md).

The arrival is often a number. It may also be the word playerx or playery, and the files sometimes add a signed offset (playery+10, playerx-15, playery-1.5). Across 18,710 arrival fields in the archive counted by client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md, 12,035 were plain numbers (integers and halves such as 24.5), 6,652 were playerx or playery alone, 21 in 14 levels added a signed number, and 2 were a stray word. Whether the stock client evaluates the offset, reads it as zero, or keeps only the keyword is open in that note. No capture shows it. The converter keeps the spelling as written. Chapter 4 says what would settle the offset.

Destination extensions in that same count: .graal 9,203, .nw 140, .enigma 10, and no extension 2. The .enigma names and the names with no extension are unexplained there. No client from 1st Graal through 2.05 adds .zelda to a bare level name or falls back between extensions. The editor adds .graal on save (client-internals/bare-level-names.md).

Z3-V1.03 links are written with decimals, for example a box and an arrival that include 43.5. A negative arrival coordinate in those files means the player's own coordinate is kept (formats/level-binary.md). That is a different convention from the playerx / playery words in Graal files. Both are in the notes. Neither note says one client accepts the other spelling.

On the wire, server packet 1 (`LEVELLINK`) is the same text, `dest x y w h newx newy`, split on spaces, and `newx` / `newy` may be `playerx` / `playery`. `[srv]` `[2.05]` (`protocol/packet-layouts.md`). The cleaned opcode note pairs that packet with the `LINK` line of a text level (`protocol/session-opcodes-cleaned.md`).

A gmap edge, described below, is a different event. The player stays inside one named map and changes a cell index. A link changes the level name.

Baddies, NPCs, chests, and signs

A binary baddy is three signed bytes, x, y, and type, then a verse line ended by a newline. The verses are separated by backslashes: sight, hurt, attack. The list ends at `FF FF FF` and an empty line. In the Java readers for `Z3-V1.02` and `Z3-V1.03`, each baddy is a 4-byte record whose last byte is always a newline. A newline can be a legal x or y, so the three bytes come before the search for the line end. Types seen in the checked Z3 files are 0 through 6: green, blue, and red soldier, shot soldier, swamp soldier, jumping hare, octopus. That check covered 139 distinct real files (`formats/level-binary.md`).

On the wire a baddy is packet 2, a baddy id plus a property stream. An unknown id creates the baddy. The property for the type runs from 0, `graysoldier`, through 9, `dragon`. The three verse properties are the same backslash-separated line, split. `[srv]` (`protocol/player-props.md`, `protocol/packet-layouts.md`). The text file can store a type name or a number from 0 to 9. The converter will not invent the byte for a name it cannot place (`formats/glevnw01.txt`, `tools/level-convert/README.md`).

A binary NPC is a biased byte for x, a biased byte for y, then `image` and the script joined by `#`. An empty image means a script-only NPC. Inside that script, a newline is the Latin-1 section sign, byte 167 (`formats/level-binary.md`, `formats/sign-text.md`). On the wire, packet 3 carries a 3-byte id and then an NPC property stream. Ids below 1000 are level NPCs. Ids of 1000 and above are server-wide. A new NPC gets the `playerenters` event if the player is in that level. `[srv]` `[2.05]` (`protocol/packet-layouts.md`). The script property, for a pre-4.0 client, is the GS1 source text (`protocol/player-props.md`).

A binary chest is four biased bytes: x, y, item, sign index. The item index is the table in `npc-library/gs1-reference-205.md` (`formats/level-binary.md`). On the wire, packet 4 is an opened flag, then x and y. If the chest is not opened, the item and the sign index follow, and a message shorter than five bytes is dropped. The client opens a chest with packet 20, x and y in whole tiles, and the server answers with the chest marked opened. `[srv]` `[2.05]` (`protocol/packet-layouts.md`).

Signs are the record most often misread. In a `.graal` or `.zelda` file, and in server packet 5, each text byte minus 32 indexes one alphabet, shared by the 2.05 client and the server. Codes 0 through 61 are `A`–`Z`, `a`–`z`, and `0`–`9`. Codes 62 through 96 are punctuation, a space at code 89, and a newline at code 96. Several slots are `#` placeholders. One or two of them stand for a button icon, an arrow, or a similar mark. Code 86 is a literal `#`. A character outside the alphabet is written as the escape `#K(n)`. Bytes under 32 are controls and are not shown (`formats/sign-text.md`).

The editor writes a line break as the escape `#K(13)` followed by the newline code. In a raw dump that escape is the six characters `v*eUWf` plus a byte with the high bit set. It is a line break. An earlier note called it a space, and an earlier note described a second alphabet with a different offset. Both were withdrawn in `formats/sign-text.md`. Viewers that drop non-ASCII bytes are why many house-outside signs look garbled.

A `GLEVNW01` sign is already plain text between `SIGN` and `SIGNEND`. The alphabet is not applied. The same is true of NPC scripts (`formats/sign-text.md`).

Signs at `(0, 0)` are common in the levels used to test the cache decoder. They hold dialogue, so they are probably messages that a script shows by index, rather than signs standing on the map. That reading is inferred (`formats/level-binary-client.md`).

The web-level cache stores signs as x plus 32, y plus 32, the sign text, and a newline. That plus-32 is the same bias as the one-byte encoding used everywhere else (`formats/gwebl001-cache.md`).

Pictures, stamps, and a name for an animation

A stamp file is not a level. Its signature is `GOBJSET01`. Five sets ship with the client: `objectsStandard`, `objectsHouse Inside`, `objectsCastle`, `objectsSnow`, and `objectsVolcano`. Each object is a width, a height, a name that may contain spaces, then `height` lines of `width * 2` base64 characters, in the same 12-bit packing as `GLEVNW01`, closed by `OBJECTEND`. There is no layer, link, NPC, or type byte. Placing a stamp writes tile ids onto the board. Collision is decided afterwards by the tables in chapter 4. The five sets hold 288 named multi-tile objects (`formats/gobjset01.md`).

The first stamp in `objectsStandard` is the 2 by 2 Bush. Its base64 decodes to ids 2, 3, 18, and 19. That id list is the one fact the stamp note, the type table, the walk note, and the lab note share. What those four ids are called, and whether a foot may stand on them, is the first disagreement in chapter 4.

Animation is a separate file, `GANI`, described in `formats/gani.txt`. On the connection it is a name in a property. The client fetches the file the way it fetches any other asset. Before 2.1, the property slot later used for that name is the bow image (`protocol/packet-layouts.md`).

A gmap is a grid of level names

A `.gmap` file does not contain a board. The first line is `GRMAP001`. The rest is a grid of level names, plus whatever the writer wanted to remember about how the grid was built. The level bytes live in the level files. Reading a gmap does not recreate them (`formats/gmap.md`, `formats/grmap001.txt`, `tools/level-convert/README.md`).

Three writers emit the signature, and they do not emit the same keys (`formats/grmap001.txt`).

The dungeon generator writes `WIDTH`, `HEIGHT`, and `GENERATED` set to the bottom-right level name. There is no `LEVELNAMES` block. The client rebuilds the grid from that name. Level names are a start string, a column letter, and a 1-based row. Here `GENERATED` is a filename.

The terrain generator writes `WIDTH`, `HEIGHT`, and `GENERATED 1`, and here the `1` is a flag. It then writes a seed, a base, a borders flag, height and chaos, a per-level height and chaos, `MAPIMG`, one level name per row, a `HEIGHTMAP` block, and sometimes `RANDOMSEEDS`. Level names are a basename, a column letter running `A` then `AA`, a zero-padded row, and `.nw`. Each level file carries its own 9 by 9 `HEIGHTS` block. The height field on the gmap is `HEIGHTMAP`.

GraalLevelGen writes `WIDTH`, `HEIGHT`, and a `LEVELNAMES` list of quoted filenames separated by commas. That is a third shape.

The server reader knows `GRMAP001`, `WIDTH`, `HEIGHT`, `GENERATED` (stored, and not used to rebuild names), `LEVELNAMES` through `LEVELNAMESEND` as quoted comma-separated names in row order, lowercased for lookup, `MAPIMG`, `MINIMAPIMG`, `NOAUTOMAPPING`, `LOADFULLMAP`, and `LOADATSTART`. `[srv]` (`formats/gmap.md`). The

converter keeps generator lines and does not turn them into cells, and it does not expand `GENERATED`. It refuses a grid above 512 on a side. That limit is the tool's own guard (`tools/level-convert/README.md`).

`protocol/format-notes-from-salvage.md` calls a gmap a level list that carries HEIGHTS blocks and is not itself a board. Chapter 4 quotes that sentence against the three writers. `HEIGHTS` in the detailed notes belongs to the level. `HEIGHTMAP` belongs to one writer. A server can read a gmap that has neither.

A player standing on a gmap has the gmap's name in the current-level property, and the cell in two further properties, x and y. Server packet 49 carries x, y, z, then the cell x and y. Packet 35 from the client preloads a neighbour, with a 5-byte modification time that is all zeros when the level is not cached. `[srv]` (`formats/gmap.md`, `protocol/packet-layouts.md`).

A real 2.22 client crosses an edge without a special link packet. It keeps a global position, local coordinate plus 64 times the cell, and it sends the wrapped local x or y together with the changed cell property (property 43 or 44) and a fresh tamper-check (client packet 39). `[cap]` (`formats/gmap.md`). That send is the walk message of the last section, with one extra property.

2.05 contains the `GRMAP001` string, and a reader that knows `WIDTH`, `HEIGHT`, `GENERATED`, the generator keys, `MAPIMG`, `HEIGHTMAP`, `RANDOMSEEDS`, and `HEIGHTS`, plus a map class. No static caller of that reader was found, so it may be dormant. 2.05 has no handler for packet 49, and it has no property name for the gmap cell. An earlier version of `formats/gmap.md` said 2.05 had no `GRMAP001` string at all. That was withdrawn in the same note. The path that note treats as live in 2.05 is the text overlay: `bigmap.txt` with `map.png`, and `minimap.txt` with `map2.png`. A cell is a level file name, and `x` marks an empty cell. The column count is the number of cells on the first line. The shipped `bigmap.txt` is 20 columns by 21 rows of quoted names. In the offline tutorial the client switches to `tutorialmap.txt` and `tutorialmap.png`. A server-list entry with extra fields can replace the file names and two pairs of integers. Those integers are probably an offset or a scale. Their use is not traced. `[2.05]` `[inf]` (`formats/gmap.md`).

The 2.05 executable also carries the offline tutorial as 13 resources, all plain `GR-V1.03`, named `tutorial_a-1.graal` through `tutorial_c-4.graal`, including `tutorial_b-2_house.graal`. When the client is not connected and the requested name starts with `tutorial`, it opens the matching resource instead of a file on disk. The levels form a 3 by 4 grid joined by edge links. The house, the B4 level, and the B2 level hold all the NPCs, chests, and signs. `[2.05]` `[test]` (`formats/level-binary-client.md`).

The copy kept beside the client

A 2.05 client can skip a full download by keeping a local copy, keyed by the level's modification time. The file is `GWEBL001`. It is written when the server sends packet 39, the 5-byte modification time. It is read when the client enters a level, if the file exists. A file whose header does not match the current server is deleted. `[2.05]` (`formats/gwebl001-cache.md`).

The path is under the executable, in `weblevels`, then a directory for the server host, then the level name in lower case with any path removed, a hyphen, the server port, and `.code`. The reader also tries the name without the port. A real example from the test set is `bank.nw` beside port 14803, under a placeholder host. `[test]`

On disk the plaintext length is padded, the body is DES in 8-byte blocks with a fresh key each block, and an 8-byte checksum follows. The key stream is the Delphi random generator, seeded from a fixed start value multiplied by each byte of the lowercased level name. The encryption therefore needs only the level name. A wrong name fails the checksum. `[2.05]` `[test]` The same note records that all 144 files in a 2.22 install decrypted, parsed, and re-encrypted byte for byte.

The plaintext starts with `GWEBL001`, the server host, a server signature, the 5-byte modification time, the level name, and then an ordinary GR-V body: `GR-V1.03` in 2.05, or `GR-V1.03` / `GR-V1.04` in 2.31. The signature matches packet 25, or the stored value 1 matches a server value of 73. All 144 test files hold 73. The modification time, read as a 35-bit integer, is a Unix time. `[2.05]` `[test]`

The cached body is selective. The board, the links, and the signs are stored. Baddies are only the terminator. NPCs and chests are empty. Heights appear in 2.31, and they are what switches the magic to `GR-V1.04` (`formats/gwebl001-cache.md`). A cache hit is why the client can ask for changes instead of a full board. The request that carries the cached time is client packet 30, below.

How the bytes stay printable

From the 1999 servers onward, a message is a one-byte id plus a payload plus a newline. Several messages are concatenated and then wrapped in one frame: a 16-bit length, most significant byte first, then that many bytes of body. The id on the wire is the logical id plus 32, so the id byte stays in the printable range (`protocol/codec-and-framing.md`, `protocol/gserv-1999.md`).

Small integers use the same bias. A `GCHAR` is one byte, the value plus 32, and it can hold 0 through 223. A `GSHORT` is two bytes, seven bits each, high group first, each plus 32, from 0 through 28,767. Longer forms take three, four, or five such groups. A `GSTRING` is a `GCHAR` length and then that many bytes, so a string tops out at 223 bytes. The 2.05 sender refuses a payload that already contains a raw newline. `[srv]` `[2.05]` (`protocol/codec-and-framing.md`).

The frame length is not a `GCHAR`. A tile inside a full-board packet is not one either. When a board or a file must travel, packet 100 announces a raw length, and the next that many bytes are one message even if they contain newlines. The server puts packet 101 (the board) and packet 102 (a file) behind that announcement. 2.05 handles 100 in its main table and does not handle 101 or 102 as their own cases, which fits a world where those two only arrive behind 100. `[srv]` `[2.05]` (`protocol/codec-and-framing.md`).

For a 2.05 client the body is zlib. Messages collect in an output buffer, the buffer is compressed, and it is flushed at 4,096 bytes or at the end of the tick. If more than 256 KB is queued unsent, the client drops the connection. The 2.05 client links zlib 1.1.2. `[2.05]` Server-to-client traffic for this generation is zlib with no extra cipher byte. `[srv]` `[cap]` Zlib bodies in the cleaned opcode note begin with the bytes 120, 156 or 120, 218 (`protocol/session-opcodes-cleaned.md`). The login-frame note assigns those headers: 120, 218 is the best compression level, which the real clients use, and 120, 156 is the default level, which one later implementation uses. The deflate body and the checksum for the same plaintext still match (`protocol/login-frames-by-version-2026-10-25.md`).

The cipher generation is independent of the level-file generation.

Connection	What a frame is	Tag
Generation 1, first Graal through 1.1.9	A text line. No length, no zlib, no login token.	`protocol/relay-1998.md`
Generation 2, 1.21 onward, and a type-0 client whose first eight bytes are already a known token	Length plus zlib. No cipher.	`protocol/gserv-1999.md`, `protocol/codec-and-framing.md`
Generation 3, including 2.05 and 2.17	Length plus zlib. After the login line, one extra byte is inserted into each message.	`protocol/codec-and-framing.md`
Generation 4, client type 4 (2.19 through 2.21)	bz2, then a partial XOR. The client side of this generation was not in the folder that was read.	`[srv]`
Generation 5, client type 5 (2.22 and later) and RC2	A compression byte, then the same partial XOR. The login frame itself is still plain zlib, with no compression byte and no XOR.	`[srv]` `[bin]`

Connection	What a frame is	Tag
Generation 6	Named for v6. Not implemented in the server code the codec note summarises.	`[srv]`

Generation 3 is the one a reader of 2.05 undoes. The iterator starts at the constant `0x04A80B38`. The key is a 7-bit random value, sent once in the login line. From the second message on, including the rest of the first frame, the client steps `iterator * 0x08088405 + key`, wrapping at 32 bits. The position is one plus the low 16 bits of the iterator, modulo the message length. That length includes the trailing newline, so the position is 1-based and can be 1, which puts the extra byte in front of the id. The inserted byte is a biased value taken from the high half of the iterator. A decoder that assumes the id is the first byte will misread the messages where the extra byte came first. The inserted byte is never a newline, and it never lands after the terminating newline. `[2.05]` `[cap]` (`protocol/codec-and-framing.md`).

The server splits off the line, which drops the newline and keeps the extra byte, and deletes at the 0-based index given by the same low 16 bits modulo that length. The note's point is that this index is the same place the client used. A check of three 2.05 sessions found the expected byte at the predicted index on every client-to-server message after the login line, including messages where it sat at index 0. There were no mismatches. `[cap]` The server's own generator inserts a fixed parenthesis rather than an iterator-derived byte. The receiver throws the byte away. `[srv]` Generation 3 runs on each message after decompression, and it runs only from client to server. Server-to-client traffic stays plain zlib. `[srv]` `[cap]`

The login line is the clean line. The language message that follows it in the same first frame already carries the extra byte. An older note that put the extra byte on the first line, and an older note that called the key a random pad, were both corrected. The key is the second `GCHAR` of the login (`protocol/codec-and-framing.md`, `protocol/login-and-walk.md`).

Generation 5 uses the same iterator and key, and XORs 4-byte groups of the body. A short last group still steps the iterator and is XORed a byte at a time. The cap is 12 groups when the body is uncompressed and 4 groups when it is zlib or bz2. An earlier note counted only complete groups of a compressed body. That count desynchronises every later frame. The corrected count is one step per group present, including a partial tail. Evidence named in the codec note: the server routine, 27 of 27 server-to-client frames from four 2.22 game servers plus both listserver frames, and the lab sessions against the lab server. On the client send side, a real 2.22 client under Wine sent 580 uncompressed frames and 17 zlib frames in the sessions that were counted. All 17 had a length that was not a multiple of 4. All 17 decode under the corrected rule. No bz2 frame was sent by that client in those sessions. A client body of 4,096 bytes or more, which is what would select bz2, had not been seen. `[srv]` `[cap]` `[2.22]` (`protocol/codec-and-framing.md`).

2.22 and 2.31 choose the compression byte by the size of the buffer about to be flushed: under 48 bytes, uncompressed unless zlib is actually shorter; from 48 bytes up to but not including 4,096, zlib; at 4,096 and above, bz2. The length field counts the compression byte. The login frame is flushed first, in the fixed zlib mode, and only then does the socket switch. `[bin]` (`protocol/client-versions.md`).

`protocol/PROTOCOL-CHEATSHEET.md` is an older one-page sketch of this machinery. It still prints a mystery raw byte, an RC client with no cipher, different nicknames for packets 100, 101, and 102, and a 1,024-byte send slice. `protocol/login-recovery-start.md` still prints guessed login ids. Chapter 4 quotes those lines beside the layout this section uses, which is `protocol/login-and-walk.md` and `protocol/codec-and-framing.md`. The direction order south, west, east, north is withdrawn in `protocol/login-and-walk.md`. The numbering this chapter uses is the one that note keeps: 0 up, 1 left, 2 down, 3 right.

How a session starts

The first Graal clients do not have this frame. From the first Delphi beta through 1.1.9, the client opens a TCP connection to a lobby. The 1998 binaries default to the loopback address on port 14801. Later connect dialogs name a relay on port 14802. Every message is a text line, and the first character is the command. The read buffer is 1 KB. The lobby groups connections into named games, assigns an 8-character id, and forwards lines that begin with `D`. Levels are not on this socket. A separate small client fetches them with HTTP. The port comes from the URL (`protocol/relay-1998.md`).

Joining is the line `N`, the game name, `#`, and a name. The clients send the name `def` and send the real nickname later, inside a `D` line. The server answers `I` and an 8-character id. A `P` line asks to play with someone, or accepts. Once a partnership is confirmed, the client pushes its state to the peer: position, colours, nickname, sword, shield, and the rest. There is no account and no password on this connection (`protocol/relay-1998.md`).

From 1.21 the game server is the thing on the other end of the socket, and levels come from it. The frame is the length-plus-zlib frame above, with zlib 1.0.4, and every flush is compressed. There is no size threshold and no cipher. The DES code in these clients is for local files (`protocol/gserv-1999.md`, `formats/graalsav.md`).

The first message is id 0. On the wire that is a space, meaning a player. An exclamation mark, type 1, is RemoteControl. Then eight bytes of token, then the account and the password as length-biased strings. The connect dialog's defaults are `guest` and `guest`. The tokens are `GSERV007` (1.21), `GSERV011` (RemoteControl 1.28 and 1.3), `GSERV013` (1.28 beta 2), `GSERV014` (1.3 test), `GSERV016` (1.31), and `GSERV018` (1.32). They are a protocol revision. Tokens that begin with `GNW` start at 1.33 (`protocol/gserv-1999.md`).

The 1.21 server accepts `GSERV007` only. The 1.28 server accepts through `GSERV013`. The 1.3 server accepts through `GSERV014`. A newer token is refused with the packet later called the disconnect message, id 16, and a sentence that names the version the server wanted. A captured 1.3 server answered `GSERV016` and `GSERV018` that way. `[cap]` Login failures (disabled account, account already in use, unknown account) use that same id (`protocol/gserv-1999.md`).

The server-to-client numbering used through 2.x is already the numbering in 1.21. Id 1 is a link, 2 is a baddy, 3 is an NPC, 4 is a chest, 5 is a sign, 7 is a board change, 8 is another player's properties. A good player login on the 1.3 server, captured under Wine, sends own properties (9), flags if the account has any (28), a warp (14), raw-data (100) and then a separate frame holding the board (101), the level name (6), links (1), signs (5), the modification time (39), NPC properties (3), and the leader flag (10). The 1.28 server sends that sequence without 39. The 1.21 server also sends no 39, and it packs its shorter reply into a single zlib frame. When someone is already on the level, the newcomer gets that player's properties just after its own. `[cap]` (`protocol/gserv-1999.md`).

The 2.05 game-server login is the same shape with a key byte inserted, and with a `GNW` token (`protocol/login-and-walk.md`).

1. A `GCHAR` type. The value 0 is a player, 1 is RemoteControl, 2 is an NPC-server, 3 is NC, 4 is a client from 2.19 through 2.21, 5 is a client from 2.22 on, 6 is RC2, and 8 is the web client. The server also keeps a bit, one shifted left by that value (`protocol/login-and-walk.md`).
2. For type 0, a `GCHAR` key, `rand` masked to 7 bits. This is the generation-3 key.
3. Eight token bytes. For 2.05 they are `GNW03081`, built one character at a time, so there is no string literal to search for.
4. The account and the password, each a `GSTRING`.
5. A newline.

2.05 puts the key before the token. A server that expects a known token in the first eight bytes will not recognise them, rewinds one byte, switches to generation 3, reads the key, and then reads the real token. [srv] (protocol/login-and-walk.md). Current server tables map GNW03081 to a version enum that sits below the 2.1 enum, so 2.05 takes the pre-2.1 paths: no file modification times on the file-transfer layout, the bow slot where a later client puts an animation name, the older level sequence, and forwarding of the global gmap position. [srv]

There is no identity string after the password in 2.05, 2.17, 2.22, or 2.31. A static read of those login routines found none. The server's optional identity-string read is for later clients. [2.05] [bin] (protocol/login-and-walk.md, protocol/client-versions.md).

Right after the login line, 2.05 sends client packet 37, the language, with English observed. [2.05] [cap] From 2.17 the next message is packet 39, the tamper check: 10 raw bytes, constant per client build, two CRC-32 words over the client's own image. The bytes can sit below 32, which a biased integer byte cannot, and none of them is a newline. 2.05 does not send it. 2.31 re-sends it about every 3.2 seconds. [bin] [cap] (protocol/client-versions.md).

The four clients that were compared side by side log in like this. [bin] (protocol/client-versions.md).

	2.05 rev 2	**2.17**	**2.22**	**2.31**
Type byte	0	0	5	5
Token	GNW03081	GNW22122	GNW03014	GNW28015
Cipher after login	generation 3	generation 3	generation 5	generation 5
Next messages	language	language, tamper check	language, tamper check	language, tamper check

2.17 still uses the 2.05 cipher. The generation-4 client, type 4, is the 2.19 through 2.21 range, and that client side was not in the set that was read (protocol/client-versions.md).

Before any of that, most of these clients ask a listserver which games exist. That login is not the game-server login, and the game token does not go there. protocol/login-frames-by-version-2026-10-25.md records three shapes, and no further per-build difference inside a shape.

- 2.00 and 2.05 send one plain zlib frame: list id 0 and the list name newmain, then they close. No account, no password, no token, no key. The 2.05 list receiver reads only ids 0 and 1. The built-in list port in those notes is 14921.
- 2.17 sends one plain zlib frame with two messages: id 0 and newmain, then id 1 with the account and the password. No token and no cipher. Its host list is arrays2.dat. The port named there is 14922.
- 2.22 and 2.31 send a first plain zlib frame of list id 7, one key byte, the token GNW30123, and newmain, then a second frame in generation 5 carrying id 1. GNW30123 is a listserver token. It is not GNW03014 or GNW28015.

2.05 has no arrays2.dat handling. From 2.17 the list address comes from that file, checked against a signature (formats/arrays-and-signatures.md, protocol/client-versions.md). Inside one generation, the observed builds send the same list bytes. No real client from 2.1 through 2.16 was observed, so the notes refuse a claim about what those builds send (protocol/login-frames-by-version-2026-10-25.md).

Chapter 3 describes the directory programs in other words: an edition line and an account line, a lab table, and later servers with their own ports. Chapter 4 sets those descriptions beside the three shapes above. This chapter does not treat them as one byte layout.

Other login tokens exist for the side connections: NC NCL11012 and NCL21075, RemoteControl GSERV023 through GSERV025, NPC-server GRNS0000 and LNX00001. [srv] (protocol/login-and-walk.md). Their

layouts live in `protocol/npc-control.md` and `protocol/chat-pm-rc.md`. Chapter 3 is the staff window and the script host. They are not how a player enters a level.

How the level arrives

Once the game server has accepted the player, it still has to put a board under their feet. Real 2.05 also waits for the message that names the active level. What a server has to send for that wait is in chapter 3 (`tools/lab-setup.md`, `server-notes/lab-servers-2026-10-30.md`).

The client asks with packet 0: a half-tile x, a half-tile y, and the level name as the rest of the line. If it already has the level cached, in memory or in the web-level cache, 2.05 sends packet 30 instead: the 5-byte modification time, then the same x, y, and name, so the server can answer with only what changed. Packet 35 asks for a gmap neighbour. `[srv]` `[2.05]` (`protocol/packet-layouts.md`). A warp to the level the player is already in, in most states, just moves the player and sends nothing. `[2.05]`

For a pre-2.1 client the server's level sequence is the one named `sendLevel141`. `[srv]` (`protocol/packet-layouts.md`).

1. If the client's cached modification time matches, the server sends the board changes since then, and nothing else.
2. Otherwise it sends raw-data plus the full board, then the level name (on the first level only), every link, every sign, and the modification time.
3. Then the board changes, every chest, every horse, every baddy, the leader flag if this client runs the level's baddies, the world time, every NPC, and the other players.

An empty board packet (server id 0) means the client should use the board it already has. If a warp to this level is still pending, 2.05 finishes the warp and then applies the runs. `[srv]` `[2.05]` On a level entry that did send a full board, an empty id 0 followed the board packet and the modification time. `[cap]` (`protocol/packet-layouts.md`).

The full board, packet 101, is 4,096 little-endian 16-bit tile ids, 8,192 bytes, behind packet 100. The raw length the server puts on packet 100 is 8,194, which is the id byte, those 8,192 bytes, and the newline. `[srv]` (`protocol/packet-layouts.md`). This is the wire form of a Graal board: a 64 by 64 array of 2-byte tiles. The `.graal` file is the bit stream above.

A partial change, server packet 7 or the client's packet 1, is a run of whole-tile `x y w h` as biased bytes, then `w * h` tile ids as `GSHORT`s, row by row. Server packet 0 can carry several such runs back to back. Parsing stops when fewer than four bytes remain or a run is shorter than it claims. An earlier layout said those header bytes were raw and the tiles were raw 16-bit words. The header bytes are biased, and packet 101 is the packet that carries raw 16-bit tiles. `[srv]` `[2.05]` (`protocol/packet-layouts.md`).

2.05 keeps two level pointers. Every level name sets the named level. NPC packets go to the object level. Packet 156 sets the object level. When there is no NPC server, each level name sets the object level too. Handlers for bombs, arrows, items, and the other level objects act only when the player is standing in the named level. `[2.05]` (`protocol/packet-layouts.md`).

The 2.05 receiver handles 59 server ids. Anything else, including 26 and the range 49 through 59, hits a silent default (`protocol/codec-and-framing.md`). Later clients add the gmap warp (49), the large-file pair, the pushed big map and minimap, the height packet 185, and, from 2.22, a second board-modify that carries a gmap cell as well as a rectangle. Counts in `protocol/client-versions.md`: 2.05 handles 59, 2.17 handles 79, 2.22 handles 80, and 2.31 handles 81. Unknown ids are ignored.

A file that is not the board (an animation, an image, a script the level named) is a different request, client packet 23, the filename as the rest of the line. Before 2.1 the server appends `.gif` when the name has no extension. The reply is packet 102 behind packet 100. For 2.05 that body is the id, a length-biased filename, and the bytes, with no modification time and no trailing newline. From 2.1 the modification time is inserted. From 2.14 a file can be split, and a file over 64,000 bytes is refused to an older client. Which of these layouts a client receives follows the server's token table. A token the server does not know is treated as older than 2.1. `[srv]` (`protocol/file-transfer.md`).

How a walk is framed

A step is not its own packet. The short client messages that begin with the character for id 2 are `PLAYERPROPS`, client packet 2. The body is a property stream: a biased property id, then a value, repeated to the end of the line. There is no length on each property, so a reader has to know every id's width. An id the client does not know ends the stream. Properties already applied stay applied, and the rest is ignored. `[srv]` `[2.05]` (`protocol/login-and-walk.md`, `protocol/player-props.md`).

Position is property 15 for x and property 16 for y. Each is one biased byte holding the tile position times 2, so the unit is a half tile. Sprite is property 17. The direction is that value modulo 4: 0 up, 1 left, 2 down, 3 right. In 2.05 the sprite byte is the direction plus 4 times an animation index, and the value 128 appears in one special state. `[srv]` `[2.05]` (`protocol/player-props.md`, `protocol/login-and-walk.md`). Chapter 2 is how far the body moves in one tick. The distance on the wire is this half-tile byte.

One captured message in the login note is the id byte for 2, then property 15, then a byte whose value is 59. That 59 is 29.5 tiles. Another is property 16 and a value of 70, which is 35.0 tiles. A third, from a 2.17 capture, carries property 10 as the animation name `walk` and then an x. In 2.05, property 10 is the bow, not an animation name. Reading a 2.05 walk with the later meaning of slot 10 will invent a gait the client did not send. `[cap]` `[srv]` (`protocol/login-and-walk.md`).

2.05 builds these messages through a helper keyed by property id. At session start the observed group is properties 0, 11, 13, 18, 31, and 33. On attach or position the group is 42, 15, and 16. After that the client sends a per-tick difference of the properties that changed. `[2.05]` (`protocol/login-and-walk.md`). The server's copy of the player's own properties comes back as server packet 9. Another player arrives as server packet 8: a 2-byte player id, then the same kind of stream. `[srv]` `[2.05]`

From 1.28 beta 2 there is a second path for the same facts. The client binds a UDP socket and sends uncompressed messages with no newline straight at another player's address and port. In captures from that era, a walking client sent UDP id 0 to the other player every 100 ms, while the server received only the changed properties over TCP and forwarded nothing. UDP id 0 carries x, y, sprite, status, and the carried sprite, properties 15 through 19. Those are the same fields 2.05 sends in its UDP id 0. `[cap]` (`protocol/gserv-1999.md`). Player properties still go over TCP while UDP is up. That was seen in the later lab captures (`protocol/packet-layouts.md`). The UDP channel is not framed as a zlib length. `protocol/udp-p2p.md` is the 2.05 map of it.

Later clients add properties 78, 79, and 80, a pixel-precision x, y, and z. 2.05 does not know them. 2.05 knows player properties 0 through 42, plus 50 and 51, and not all of those in every stream (`protocol/player-props.md`).

A gmap crossing is this same property message with one extra property: the cell x or the cell y, plus the tamper check beside it. `[cap]` (`formats/gmap.md`).

The server also places the player with packet 14: half-tile x, half-tile y, and the level name. The warp stays pending until the level data arrives. Packet 15 is a failed warp. 2.05 ignores the text of that failure and shows a short failed-warp state. `[srv] [2.05]` (`protocol/packet-layouts.md`).

Cutting a bush uses the partial-change message above. The client sends packet 1, one rectangle of whole tiles and the replacement ids. In a lab with two 2.05 clients, the player who did not own the bomb was the one who reported the patches, eight messages in one frame, and the server echoed each rectangle back to both clients as packet 7. GServer-v2 then restored the original tiles 15.0 seconds later. The 1999 server did not echo a change to the sender. Clients do not request the respawn. `[cap] [2.05] [inf]` (`protocol/packet-layouts.md`). The reason one client reported the patches is not settled in that note. Who rolls a drop, and the 1999 server's wait, are in chapter 2. Later labs record their own waits there. Those waits are that lab server's setting.

The families, named once

Z3-V1. A board, links, baddies, and signs. No NPCs and no chests. The server note gives `Z3-V1.02` 11-bit codes and a 4-byte baddy, and gives `.03` and `.04` 12-bit codes. `.03` has decimal link numbers and no verse line. `.04` adds the verse line. A negative arrival keeps the player's coordinate.

GR-V1. The same bit stream, then NPCs from `GR-V1.00` and chests from `GR-V1.01`. Twelve bits in `.00` and `.01`, thirteen bits from `.02`, which is what lets a tile id reach 4095. The 2.05 writer emits `.03`. Arrivals use numbers or `playerx` / `playery`. `GR-V1.04`, in 2.31's cache, adds the 9 by 9 height block.

GLEVNW01. The same lists as text. The board is 12-bit indices in base64, one horizontal run per `BOARD` line, layer ignored. Signs are plain text.

Gmap. `GRMAP001` names a grid. It does not store the levels. On the wire from 2.17, the player stands in the gmap and sends a cell, using the walk message. 2.05 has the parser. No caller of it was found. 2.05 shows `bigmap.txt` instead.

The package after the gmap. `tools/level-convert/SEIHAI-FORMAT.md` describes a text wrapper, `SEIHAI01`, with a kind of `nw` or `gmap` and then the payload bytes unchanged. It is marked inferred. A 2.22 client does not read it, and it is not in a client string table. `formats/seihai-boards.md` is a further thing: a small game whose files reuse the old extensions as aliases (`.seihai` beside `.graal`, `.as` beside `.nw`, headers `SH-V1` and `GLEVAS01`). Its tile ids 0 through 18 are names in that game, grass through a heal tile, and its water is solid. Those ids are not `pics1` indices. This chapter does not treat that game as a Graal generation.

Connection. Generation 1 is a lobby line, and the level comes from HTTP. Generation 2 is zlib, a `GSERV` token, account and password, and the level comes from the game server, already under the packet numbers 2.x still uses. Generation 3 adds the key byte and the inserted byte, and it is the 2.05 and 2.17 game login. Generations 4 and 5 change the compression and add a partial XOR. The login frame of a generation-5 client stays plain zlib so the key can be read. The listserver, beside all of this, is the three shapes above, and the other wordings in chapter 4.